



**SOMAIYA**  
VIDYAVIHAR UNIVERSITY

## **Mini Project**

***Automated CI/CD and Real-Time Failure Monitoring of a  
Containerized Environmental Sensor Dashboard***

**Github Link:**

[https://github.com/Yashvi2874/DevOps\\_failure\\_monitoring](https://github.com/Yashvi2874/DevOps_failure_monitoring)

by

**16010123318**

**Aditi Agrawal**

**16010123328**

**Shubhpreet Kaur**

**16010123341**

**Yashasvi Gupta**

For the subject

**DevOps**

Department of Computer Engineering  
Academic Year 2026-27

# Table of Contents

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>Abstract.....</b>                                       | <b>3</b>  |
| <b>1 Introduction.....</b>                                 | <b>4</b>  |
| 1.1 Background .....                                       | 4         |
| 1.2 Problem Statement .....                                | 4         |
| 1.3 Objectives .....                                       | 4         |
| 1.4 Scope .....                                            | 4         |
| <b>2 Tools and Technologies.....</b>                       | <b>5</b>  |
| 2.1 Technology Stack .....                                 | 5         |
| 2.2 Why These Tools.....                                   | 5         |
| <b>3 System Architecture.....</b>                          | <b>6</b>  |
| 3.1 Overview .....                                         | 6         |
| 3.2 Components.....                                        | 6         |
| 3.3 End-to-End Flow .....                                  | 7         |
| <b>4 Implementation .....</b>                              | <b>8</b>  |
| 4.1 The Application.....                                   | 8         |
| 4.2 Metrics Exposed .....                                  | 8         |
| 4.3 Version Control Workflow .....                         | 9         |
| 4.4 Containerization .....                                 | 9         |
| 4.5 Jenkins Setup as Code .....                            | 10        |
| 4.6 CI/CD Pipeline .....                                   | 11        |
| 4.7 GitHub Actions.....                                    | 12        |
| <b>5 Monitoring and Alerting .....</b>                     | <b>14</b> |
| 5.1 Metric Collection with Prometheus.....                 | 14        |
| 5.2 Alert Rules.....                                       | 14        |
| 5.3 Routing, Grouping and Inhibition in Alertmanager ..... | 15        |
| 5.4 Alert Receiver.....                                    | 16        |
| <b>6 Testing.....</b>                                      | <b>18</b> |
| 6.1 Testing Strategy.....                                  | 18        |
| 6.2 Testing the Alert Rules.....                           | 18        |
| 6.3 Smoke Test .....                                       | 19        |
| <b>7 Results .....</b>                                     | <b>20</b> |
| 7.1 Pipeline Runs.....                                     | 20        |
| 7.2 Failure Detection .....                                | 21        |
| <b>8 Critical Analysis .....</b>                           | <b>24</b> |
| 8.1 Challenges Faced and Solutions.....                    | 24        |
| 8.2 Design Trade-offs.....                                 | 24        |
| 8.3 Limitations.....                                       | 24        |
| 8.4 Future Improvements.....                               | 25        |
| <b>Conclusion .....</b>                                    | <b>26</b> |
| <b>References.....</b>                                     | <b>27</b> |

## Abstract

Software teams have to ship changes often without breaking what people rely on, and they should hear about failures before their users do. This project applies the DevOps practices of **version control, continuous integration, continuous delivery, containerization, monitoring and alerting** to a small but realistic application: a web dashboard showing temperature, humidity, CO<sub>2</sub> and PM2.5 readings from three environmental sensors.

Every change pushed to GitHub is detected by **Jenkins**, which runs the unit tests inside a **Docker** container, validates the monitoring configuration, builds a versioned image, deploys it with Docker Compose and smoke-tests the deployed system. Code that fails its tests is never deployed, and a release that fails the post-deployment health check is rolled back automatically to the last good image. The running application is observed by **Prometheus**, which scrapes its metrics every five seconds and evaluates seven alert rules covering application availability, API errors, restarts and abnormal sensor readings. **Alertmanager groups and routes the resulting alerts and passes them to** a webhook receiver that stands in for Slack, e-mail or a pager. Pipeline failures are reported through the same channel.

The system was evaluated by injecting real failures: overheating, offline and polluted sensors, an API returning errors, a crashed process, a stopped container, a commit with a failing test and a deliberately broken release. In every case the failure was either blocked before deployment or reported within roughly 30 to 55 seconds, and a resolved notification was sent once the problem was fixed. The report also discusses the design trade-offs, the challenges faced during implementation, and the changes required to take the system to production.

# Introduction

## 1.1 Background

DevOps closes the gap between writing software and running it. Instead of someone copying files to a server by hand and hoping nothing breaks, every step from commit to running system is written down as code and carried out by machines. Once the system is live it is measured all the time, so problems show up in monitoring before they reach users.

The DevOps lifecycle is commonly described as a continuous loop:

**Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → (back to Plan)**

This project covers the Code-to-Monitor part of that loop from start to finish. The application itself is deliberately simple, because the focus of the project is the **delivery pipeline and the monitoring around the application**, not the sensor hardware. The sensor readings are simulated, which also makes it possible to break things on purpose and observe how the system reacts.

## 1.2 Problem Statement

Without automation, a team running an application like this faces three recurring problems:

1. **Untested code reaches users.** Someone forgets to run the tests before deploying, and a defect goes live.
2. **"It works on my machine."** The application behaves differently on the developer's laptop, the tester's machine and the server, because each has different library and runtime versions.
3. **Failures are noticed late.** A sensor stops reporting, the server room overheats or the API starts failing, and nobody finds out until someone happens to look at the dashboard.

The aim is a system where every change is tested and deployed automatically, a bad release undoes itself, and every failure produces a clear notification quickly.

## 1.3 Objectives

1. Maintain all application code, infrastructure definitions and configuration in Git, using feature branches.
2. Package the application as a Docker image so that it runs identically on every machine.
3. Run automated linting and unit tests on every change and stop the pipeline if any of them fail.
4. Deploy automatically with Jenkins, verify the deployment with a smoke test, and roll back automatically if the new version is unhealthy.
5. Expose application and sensor metrics and collect them with Prometheus.
6. Define alert rules for application and sensor failures, and deliver alerts through Alertmanager with grouping, routing and noise suppression.
7. Demonstrate each failure scenario and measure how long detection and notification take.

## 1.4 Scope

The project runs entirely on a single developer machine using Docker Desktop. Jenkins, the application and the monitoring stack each run in containers. Real sensor hardware, cloud deployment and container orchestration (Kubernetes) are outside the scope and are discussed under future improvements.

# Tools and Technologies

## 2.1 Technology Stack

*Table 1 Tools used in the project*

| Area            | Tool (version)                                                             | Purpose in the project                                                                     |
|-----------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Version control | Git, GitHub                                                                | Single source of truth for code and configuration; Jenkins and GitHub Actions read from it |
| Application     | Python 3.12, Flask 3.1, gunicorn 23                                        | Dashboard web application, JSON API and health endpoint                                    |
| Metrics library | prometheus_client 0.23                                                     | Exposes application and sensor metrics in Prometheus format                                |
| Testing         | pytest 8.4, pytest-cov, flake8 7.3, promtool, unittest                     | Unit tests, coverage, linting, and tests for the alert rules themselves                    |
| Containers      | Docker Engine 29, Docker Compose                                           | Packaging and running all five services                                                    |
| CI/CD           | Jenkins 2.580.1 LTS (declarative pipeline, Configuration as Code, Job DSL) | Automates test → build → deploy → verify → roll back                                       |
| CI on GitHub    | GitHub Actions                                                             | Independent second check on every push and pull request                                    |
| Monitoring      | Prometheus 3.13.4                                                          | Pull-based metric collection, storage and alert rule evaluation                            |
| Alerting        | Alertmanager 0.34.1                                                        | Grouping, routing, inhibition and delivery of alerts                                       |

## 2.2 Why These Tools

- **Jenkins** is the most widely deployed open-source automation server. Its declarative `Jenkinsfile` keeps the pipeline in the repository alongside the code (*pipeline as code*), and Configuration as Code lets the whole Jenkins server be rebuilt from files.
- **Docker** removes the "works on my machine" problem: the same image is tested, deployed and rolled back, so what was tested is exactly what runs.
- **Prometheus** uses a *pull* model: it scrapes each target on a schedule. A failed scrape is itself a signal, which is how the system detects that the application is down.
- **Alertmanager** separates *detecting* a problem from *notifying* people about it, so routing, grouping and silencing can be changed without touching the alert rules.

# System Architecture

## 3.1 Overview

The system has two halves. The **delivery half** takes a commit from the developer to a running container. The **running half** is the deployed application together with the tools that watch it.

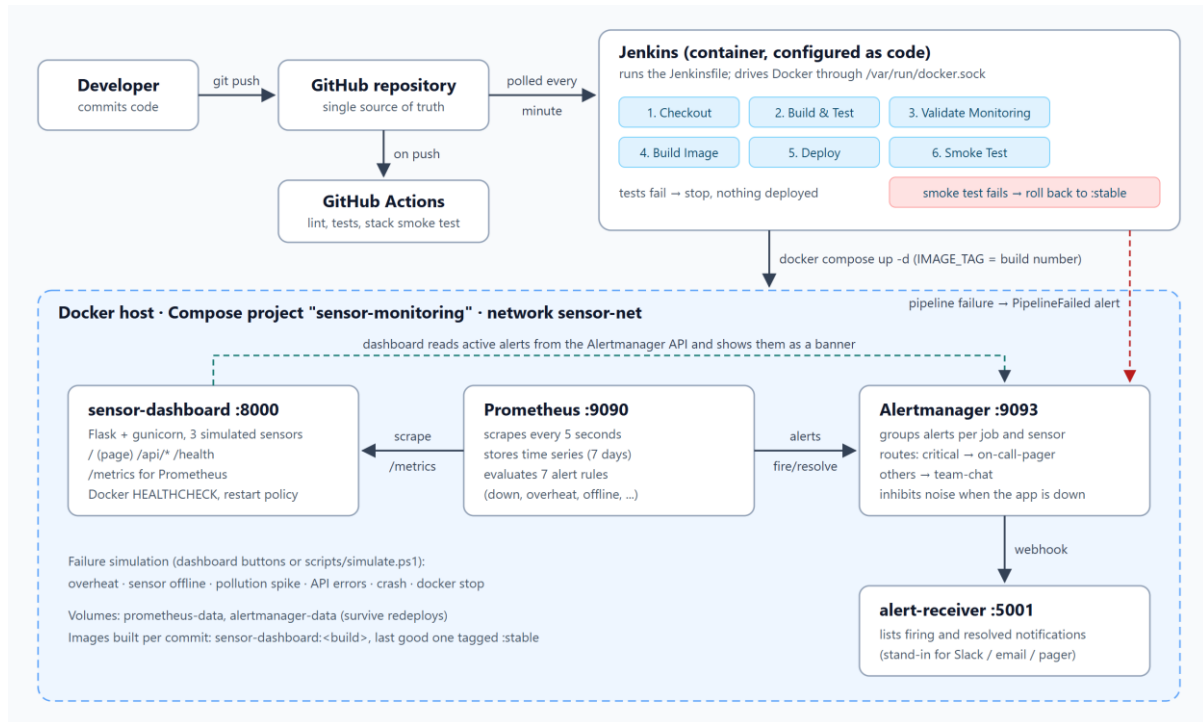


Figure 1 Overall system architecture

In the delivery half, code is pushed to GitHub. GitHub Actions runs a quick independent check, and Jenkins detects the new commit by polling the repository every minute. Jenkins runs in its own container with the Docker command-line tools installed and the host's Docker socket mounted, so every pipeline step (building images, running tests, starting containers) is carried out by the host's Docker engine.

The running half is a Docker Compose project named `sensor-monitoring` with four containers on a private network named `sensor-net`. Containers find each other by service name; for example, Prometheus scrapes `dashboard:8000`.

## 3.2 Components

Table 2 Containers and their roles

| Container        | Port | Role                                                                                    |
|------------------|------|-----------------------------------------------------------------------------------------|
| sensor-dashboard | 8000 | Flask application: dashboard page, JSON API, /health and /metrics                       |
| prometheus       | 9090 | Scrapes /metrics every 5 s, stores time series for 7 days, evaluates alert rules        |
| alertmanager     | 9093 | Receives alerts from Prometheus and Jenkins, groups, routes and suppresses them         |
| alert-receiver   | 5001 | Webhook receiver that displays every notification (stand-in for Slack / e-mail / pager) |

| Container | Port                   | Role                                                                            |
|-----------|------------------------|---------------------------------------------------------------------------------|
| jenkins   | 8081<br>(configurable) | CI/CD server; runs the Jenkinsfile and drives Docker through the mounted socket |

### 3.3 End-to-End Flow

1. A developer commits a change and pushes it to the `main` branch on GitHub.
2. Within a minute Jenkins detects the new commit and starts a build.
3. The code is linted and unit-tested inside a Docker container. If anything fails, the pipeline stops and nothing is deployed.
4. The Prometheus and Alertmanager configuration is validated and the alert rules are unit-tested.
5. A runtime image tagged with the build number is built and deployed with Docker Compose.
6. A smoke test checks that the new container is healthy, is the expected build, serves data and is being scraped by Prometheus. If it fails, the last known good image is restored automatically.
7. From then on Prometheus scrapes the application every five seconds. When an alert rule stays true for its configured duration, the alert is sent to Alertmanager, which routes it to the right receiver.
8. When the problem is fixed, the alert resolves and a resolved notification is delivered.

# Implementation

## 4.1 The Application

The application is written in Python using Flask and is served by gunicorn. Its source is in `app/sensor_dashboard/`.

### Sensor simulator

`sensors.py` maintains three sensors, each with its own baseline values: **Sensor 1** in the lab, **Sensor 2** in the server room and **Sensor 3** outdoors. A background thread advances the simulation every two seconds. Each reading follows a *mean-reverting random walk*: on every tick it moves a fraction of the way back towards its baseline and then adds a small amount of random noise. This produces slowly drifting, realistic values instead of random jumps, and the values are clamped to physical limits.

Each sensor can be switched into a failure scenario, which changes the value the reading drifts towards:

- **Overheat**: the temperature drifts towards 45 °C.
- **Offline**: the sensor stops producing readings and every attempted read is counted as an error.
- **Pollution**: CO<sub>2</sub> and PM2.5 climb towards unhealthy levels (2200 ppm and 180 µg/m<sup>3</sup>).

### Endpoints

*Table 3 Application endpoints*

| Endpoint                      | Purpose                                                                                                  |
|-------------------------------|----------------------------------------------------------------------------------------------------------|
| GET /                         | Dashboard page; refreshes itself every two seconds and shows active alerts as a banner                   |
| GET /health                   | Status, number of sensors online, uptime and the running build number; used by Docker and the smoke test |
| GET /metrics                  | All metrics in Prometheus text format                                                                    |
| GET /api/readings             | Current readings and the last 60 temperature samples for each sensor                                     |
| POST /api/simulate/<scenario> | Failure simulation: overheat, offline, pollution, api-errors, crash, reset                               |
| GET /api/alerts               | Active alerts fetched from Alertmanager's API, shown on the dashboard                                    |

## 4.2 Metrics Exposed

`metrics.py` defines the metrics on a dedicated registry. Sensor gauges are labelled with the sensor id and location; request metrics are labelled with the *route pattern* rather than the raw URL, so that requests for different static files do not each create a new time series (a common cause of high cardinality).

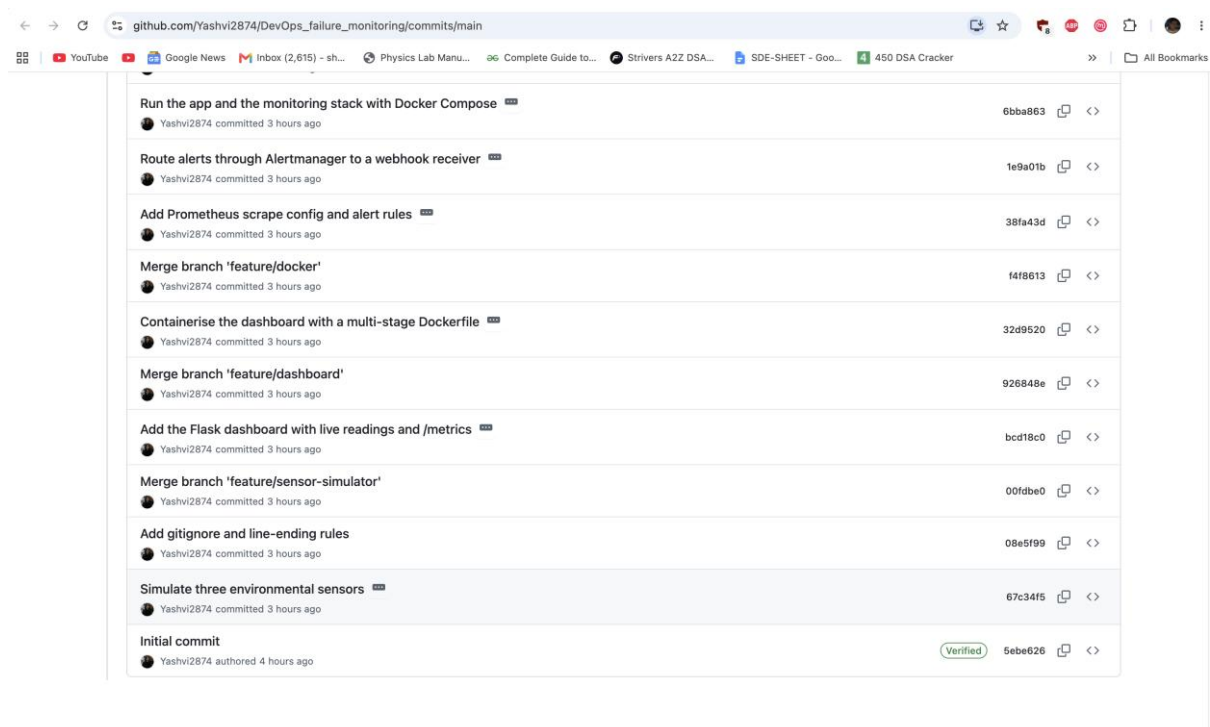
*Table 4 Metrics exposed on /metrics*

| Metric                                                                                      | Type           | Meaning                                                       |
|---------------------------------------------------------------------------------------------|----------------|---------------------------------------------------------------|
| sensor_temperature_celsius,<br>sensor_humidity_percent, sensor_co2_ppm,<br>sensor_pm25_ugm3 | Gauge          | Latest reading per sensor                                     |
| sensor_last_seen_timestamp_seconds                                                          | Gauge          | Time of the last good reading; used to detect offline sensors |
| sensor_up, sensor_read_errors_total                                                         | Gauge, Counter | Sensor status and number of failed reads                      |

| Metric                                  | Type      | Meaning                                                       |
|-----------------------------------------|-----------|---------------------------------------------------------------|
| dashboard_http_requests_total           | Counter   | Requests by method, endpoint and status code                  |
| dashboard_http_request_duration_seconds | Histogram | Request latency per endpoint                                  |
| dashboard_build_info                    | Gauge     | Which version, commit and build are running                   |
| process_start_time_seconds              | Gauge     | When the process started; used to detect crashes and redeloys |

### 4.3 Version Control Workflow

The repository is hosted on GitHub. Each feature was developed on its own branch and merged into main with a merge commit, so the history records where each feature started and ended: feature/sensor-simulator, feature/dashboard, feature/docker, feature/monitoring, feature/jenkins-pipeline, feature/github-actions, and later fix/pipeline-alerts. Secrets such as the Jenkins admin password are stored in jenkins/.env, which is listed in .gitignore and is never committed; only a .env.example template is versioned. A .gitattributes file forces Unix (LF) line endings for scripts and configuration so they behave the same when checked out on Windows.



*Figure 2 Commit history on GitHub showing the feature-branch merges*

### 4.4 Containerization

app/Dockerfile is a **multi-stage build** with three stages, so that test tools never end up in the deployed image:

**Table 5 Stages of the application Dockerfile**

| Stage   | Contents                                                        | Used for                                                |
|---------|-----------------------------------------------------------------|---------------------------------------------------------|
| base    | python:3.12-slim plus the runtime libraries                     | Shared by the other two stages                          |
| test    | base + pytest, pytest-cov and flake8 + the source and tests     | Run by Jenkins as a container to lint and test the code |
| runtime | base + only the application package, non-root user, HEALTHCHECK | The image that is deployed and rolled back              |

```
FROM python:3.12-slim AS base
COPY requirements.txt .
RUN pip install -r requirements.txt      # cached until dependencies change

FROM base AS runtime
RUN useradd --create-home --uid 10001 appuser
COPY sensor_dashboard ./sensor_dashboard
USER appuser
HEALTHCHECK --interval=10s CMD python -c "...urlopen('../health')"
CMD ["unicorn", "--workers", "1", "--threads", "4", ...]
```

Three design decisions are worth noting:

- **Layer caching:** `requirements.txt` is copied and installed before the source code, so Docker reuses the dependency layer when only the code changes. This made later builds noticeably faster.
- **Security:** the runtime image runs as an unprivileged user, and the Docker `HEALTHCHECK` lets Docker mark the container healthy or unhealthy. The Compose file sets `restart: unless-stopped`, so a crashed container is restarted automatically.
- **One gunicorn worker with four threads:** the simulator state and the metrics live in process memory. Several worker processes would each hold a different copy and Prometheus would see inconsistent values.

The Prometheus and Alertmanager configuration files are copied into small custom images instead of being mounted from the host. This allows Jenkins, which runs in its own container, to deploy the stack, and it means a change to an alert rule is built, validated and deployed exactly like a code change.

```
● (base) shubhpreetkaur@Shubhpreets-MacBook-Pro DevOps_failure_monitoring % docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
NAMES          IMAGE                                STATUS
sensor-dashboard sensor-dashboard:4                 Up 9 minutes (healthy)
prometheus      d850c63f5e1e                       Up 3 hours
alertmanager    bc3aa4bf5a6d                       Up 3 hours
alert-receiver  f0d54a6e65b6                       Up 3 hours (healthy)
jenkins         sensor-jenkins:latest              Up 3 hours
```

**Figure 3 All project containers running**

## 4.5 Jenkins Setup as Code

Jenkins itself is built from files in the `jenkins/` folder, so no manual setup wizard is needed:

- `jenkins/Dockerfile` starts from the official `jenkins/jenkins:2.580.1-lts-jdk21` image and adds the Docker CLI, the Compose plugin, Buildx and the plugins listed in `plugins.txt`.
- `casc.yaml` (**Jenkins Configuration as Code**) creates the admin user and, through the **Job DSL** plugin, the `sensor-dashboard-pipeline` job, which reads the `Jenkinsfile` from GitHub and polls for new commits every minute.

- The admin password is supplied through `jenkins/.env`. The Docker socket is shared using `group_add` instead of running Jenkins as root.

## 4.6 CI/CD Pipeline

The pipeline is a declarative `Jenkinsfile` at the root of the repository. Every step runs through Docker, so the Jenkins machine itself needs only the Docker CLI.

*Table 6 Stages of the Jenkins pipeline*

| Stage               | What it does                                                                                                                                                                                                    | If it fails                                                                             |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Checkout            | Fetches the commit and records its short hash and message on the build                                                                                                                                          | Pipeline stops                                                                          |
| Build & Test        | Builds the test image and runs flake8 and 23 pytest tests inside a container; publishes the JUnit report to Jenkins                                                                                             | Pipeline stops; nothing is deployed                                                     |
| Validate Monitoring | promtool checks the Prometheus config and runs the alert-rule unit tests; amtool checks the Alertmanager config and its routing; runs the receiver's tests                                                      | Pipeline stops                                                                          |
| Build Image         | Builds <code>sensor-dashboard:&lt;build number&gt;</code> with the version, commit and build number baked in                                                                                                    | Pipeline stops                                                                          |
| Deploy              | <code>docker compose up -d</code> with <code>IMAGE_TAG=&lt;build number&gt;</code> ; only containers whose image changed are replaced                                                                           | Pipeline stops                                                                          |
| Smoke Test          | From a throwaway container on <code>sensor-net</code> : <code>/health</code> is OK and reports this build, the API returns three sensors, <code>/metrics</code> works, Prometheus is scraping the new container | <code>rollback.sh</code> redeploys <code>sensor-dashboard:stable</code> and verifies it |

When the smoke test passes, the image is also tagged `sensor-dashboard:stable`, marking it as the last known good build. At the end of every run the pipeline reports its own result to Alertmanager: a failed build raises a `PipelineFailed` alert and the next successful build resolves it. Broken builds therefore reach the team through the same channel as application failures. A build parameter, `SIMULATE_BAD_DEPLOY`, deploys a release whose `/health` endpoint returns HTTP 503, so the automatic rollback can be demonstrated on demand.

```
stage('Smoke Test') {
    steps {
        sh 'docker run --rm -i --network sensor-net \
            -e EXPECTED_BUILD=${BUILD_NUMBER} \
            ${APP_IMAGE}:${IMAGE_TAG} python -< scripts/smoke_test.py'
    }
    post {
        success { sh 'docker tag ${APP_IMAGE}:${IMAGE_TAG} sensor-dashboard:stable' }
        failure { sh 'sh scripts/rollback.sh' }
    }
}
```

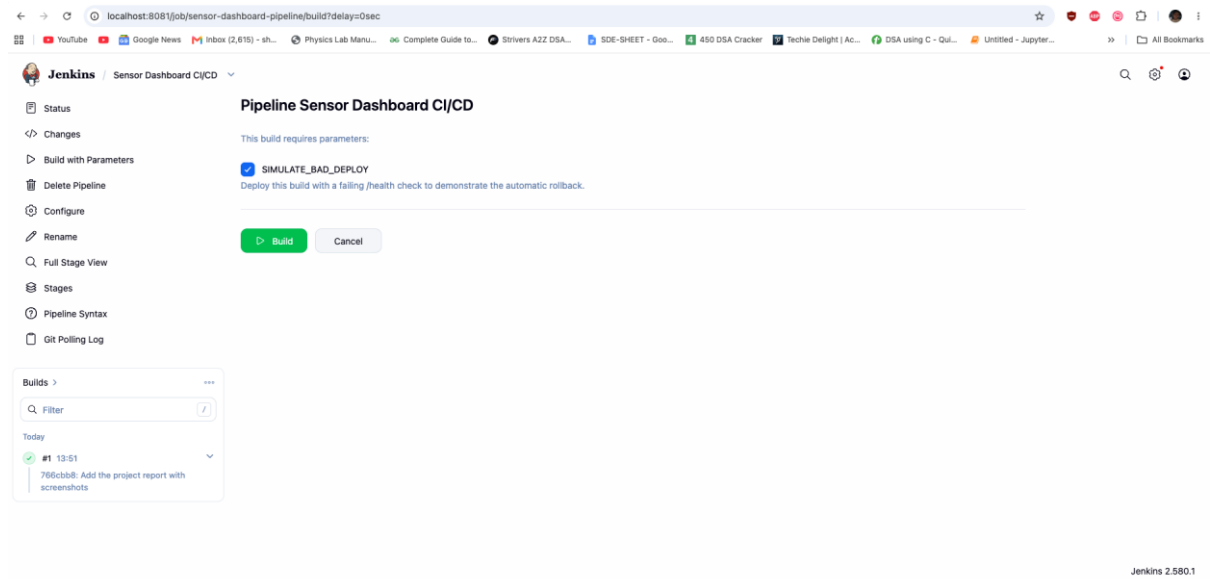


Figure 4 Triggering the rollback demonstration with the `SIMULATE_BAD_DEPLOY` build parameter

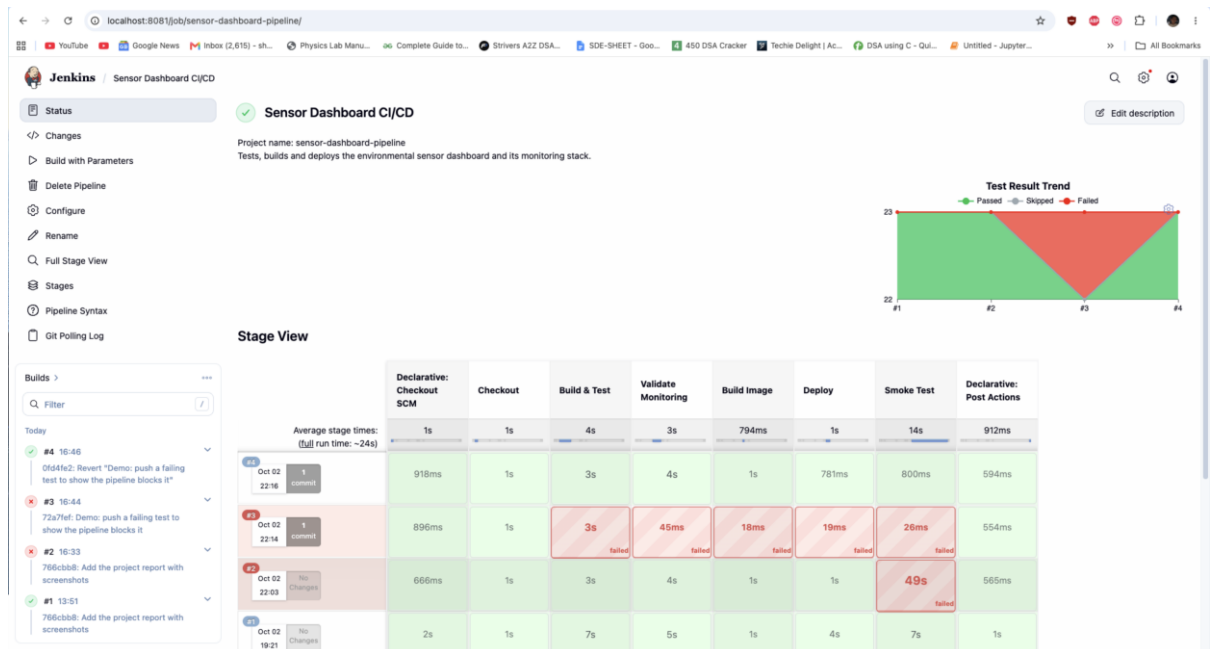


Figure 5 Jenkins stage view of the pipeline runs

## 4.7 GitHub Actions

`.github/workflows/ci.yml` runs on GitHub's own machines for every push and pull request. It lints and unit-tests the code, tests the alert receiver, builds all images, validates the Prometheus and Alertmanager configuration, starts the full stack and runs the same smoke test. This gives every commit an independent pass/fail check that is visible on GitHub even when the local Jenkins is not running.

The screenshot displays the GitHub Actions interface for the repository `Yashvi2874/DevOps_failure_monitoring`. The `Actions` tab is selected, showing a list of workflow runs. The left sidebar contains navigation options for Actions, including `All workflows`, `CI`, `Management`, `Caches`, `Attestations`, `Runners`, `Usage metrics`, and `Performance metrics`. The main area, titled `All workflows`, shows 7 workflow runs with a search filter. The runs are listed in a table with columns for Workflow, Event, Status, Branch, and Actor.

| Workflow                                                          | Event                                                            | Status  | Branch | Actor            |
|-------------------------------------------------------------------|------------------------------------------------------------------|---------|--------|------------------|
| Revert "Demo: push a failing test to show the pipeline blocks it" | CI #7: Commit <code>0fd4fe2</code> pushed by ShubhpreetKDhariwal | Success | main   | 1 minute ago     |
| Demo: push a failing test to show the pipeline blocks it          | CI #6: Commit <code>72a7fe1</code> pushed by ShubhpreetKDhariwal | Failure | main   | 4 minutes ago    |
| Add the project report with screenshots                           | CI #5: Commit <code>766cbb8</code> pushed by Yashvi2874          | Success | main   | Today at 7:14 PM |
| Revert the deliberately failing test                              | CI #4: Commit <code>aef0aea</code> pushed by Yashvi2874          | Success | main   | Today at 7:00 PM |
| Demo: push a failing test to show the pipeline blocks it          | CI #3: Commit <code>228122f</code> pushed by Yashvi2874          | Failure | main   | Today at 6:49 PM |
| Merge branch 'fix/pipeline-alerts'                                | CI #2: Commit <code>0fa0ee6</code> pushed by Yashvi2874          | Success | main   | Today at 6:47 PM |
| Merge branch 'feature/github-actions'                             | CI #1: Commit <code>d4e9057</code> pushed by Yashvi2874          | Success | main   | Today at 6:42 PM |

*Figure 6 GitHub Actions workflow runs*

# Monitoring and Alerting

## 5.1 Metric Collection with Prometheus

Prometheus is configured in `monitoring/prometheus/prometheus.yml` to scrape three targets every five seconds: the dashboard, Prometheus itself and Alertmanager. The alert rules are evaluated at the same interval. Collected data is kept for seven days on a named Docker volume, so it survives redeployments.

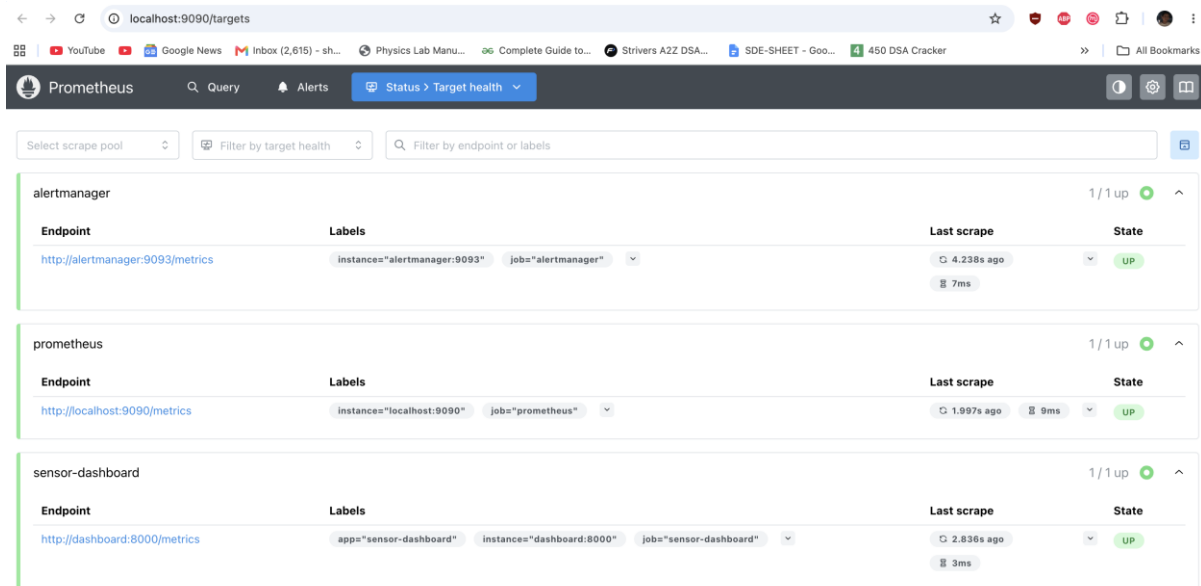


Figure 7 Prometheus target health

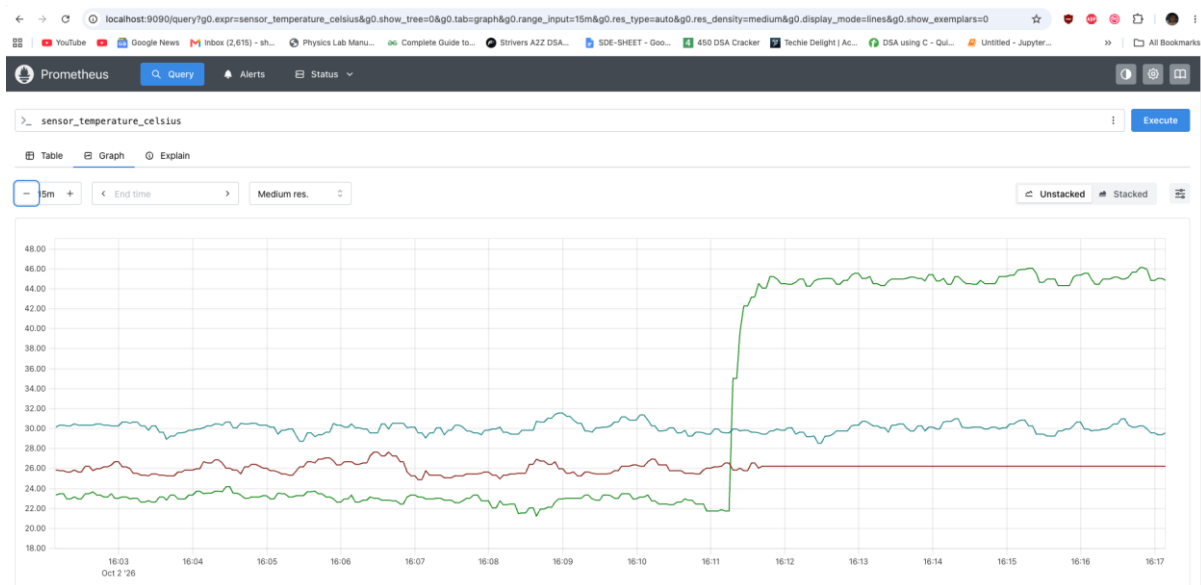


Figure 8 Temperature history in Prometheus during an overheat

## 5.2 Alert Rules

Each rule in `alert_rules.yml` is a PromQL expression plus a `for` duration: the condition must hold continuously for that long before the alert fires. This prevents notifications for momentary blips.

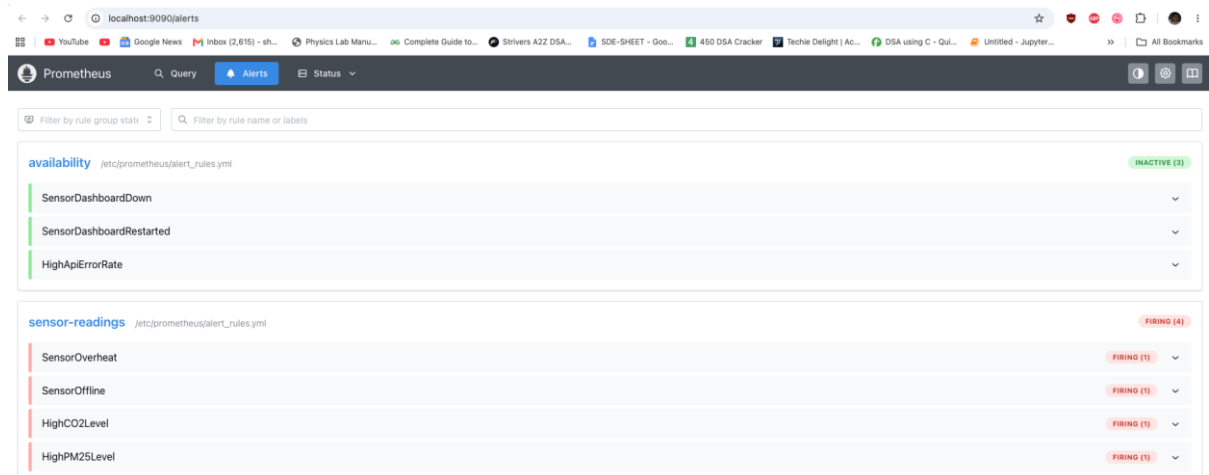
**Table 7 Alert rules**

| Alert                    | Condition                                                                | Severity |
|--------------------------|--------------------------------------------------------------------------|----------|
| SensorDashboardDown      | up == 0 for 30 s (Prometheus cannot scrape the app)                      | critical |
| HighApiErrorRate         | More than 20% of /api/* requests return 5xx, for 30 s                    | critical |
| SensorDashboardRestarted | process_start_time_seconds changed in the last 5 min (crash or redeploy) | info     |
| SensorOverheat           | Temperature > 35 °C for 15 s                                             | warning  |
| SensorOffline            | No reading for more than 10 s, for 10 s                                  | warning  |
| HighCO2Level             | CO <sub>2</sub> > 1500 ppm for 20 s                                      | warning  |
| HighPM25Level            | PM2.5 > 100 µg/m <sup>3</sup> for 20 s                                   | warning  |

For example, the error-rate rule divides the rate of failed API requests by the rate of all API requests over the last minute:

```
- alert: HighApiErrorRate
  expr: |
    sum(rate(dashboard_http_requests_total(endpoint=~"/api/.*", status=~"5..") [1m]))
    /
    sum(rate(dashboard_http_requests_total(endpoint=~"/api/.*" [1m])) > 0.2
  for: 30s
  labels: {severity: critical}
```

Offline detection deliberately uses `time() - sensor_last_seen_timestamp_seconds` rather than a simple status flag: if a sensor's last timestamp stops advancing, the sensor has gone quiet regardless of why.

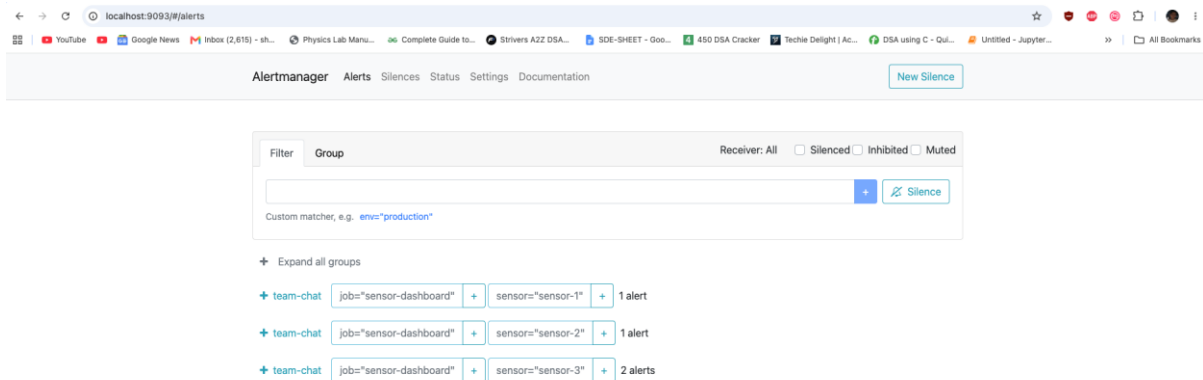


**Figure 9 Alert rules in Prometheus during an incident**

### 5.3 Routing, Grouping and Inhibition in Alertmanager

Prometheus decides *that* something is wrong; Alertmanager (monitoring/alertmanager/alertmanager.yml) decides *who* hears about it, *when* and *how often*:

- **Routing:** alerts with `severity="critical"` go to the `on-call-pager` receiver and are repeated every 30 minutes while unresolved; all other alerts go to `team-chat` and are repeated every 2 hours.
- **Grouping:** alerts are grouped by job and sensor. A pollution spike on Sensor 3 raises both `HighCO2Level` and `HighPM25Level`, but they arrive as a single notification.
- **Inhibition:** while `SensorDashboardDown` is firing, the application's warning and info alerts are suppressed, because they are symptoms of the same outage. An offline sensor's last temperature is stale, so `SensorOffline` also suppresses `SensorOverheat` for the same sensor.
- **Resolution:** `send_resolved: true` sends a second notification when a problem clears.



*Figure 10 Alertmanager grouping alerts per sensor*

## 5.4 Alert Receiver

Both receivers are webhooks pointing at the `alert-receiver` service, a small Python program using only the standard library. It stores the last 200 notifications and lists them on a web page with the channel, status, alert name, severity and summary. In a real team this would be Slack, e-mail or a paging service; only the receiver section of `alertmanager.yml` would change. The receiver can also forward every notification to a real Slack or Discord webhook if `FORWARD_WEBHOOK_URL` is set.

The screenshot shows the Alert receiver web interface at `localhost:5001`. The interface displays a table of notifications delivered by Alertmanager. The table has the following columns: Received (UTC), Channel, Status, Alert, Severity, Sensor, and Summary. The table shows 5 firing and 5 resolved alerts.

| Received (UTC)            | Channel   | Status   | Alert                    | Severity | Sensor   | Summary                                              |
|---------------------------|-----------|----------|--------------------------|----------|----------|------------------------------------------------------|
| 2026-10-02T16:18:22+00:00 | team-chat | RESOLVED | HighCO2Level             | warning  | sensor-3 | CO2 is high in Outdoors (1567 ppm)                   |
| 2026-10-02T16:18:22+00:00 | team-chat | RESOLVED | HighPM25Level            | warning  | sensor-3 | PM2.5 is high in Outdoors (117 µg/m³)                |
| 2026-10-02T16:18:07+00:00 | team-chat | RESOLVED | SensorOverheat           | warning  | sensor-1 | Lab is overheating (37.8 °C)                         |
| 2026-10-02T16:18:02+00:00 | team-chat | RESOLVED | SensorOffline            | warning  | sensor-2 | sensor-2 (Server room) stopped reporting             |
| 2026-10-02T16:12:22+00:00 | team-chat | FIRING   | HighCO2Level             | warning  | sensor-3 | CO2 is high in Outdoors (2189 ppm)                   |
| 2026-10-02T16:12:22+00:00 | team-chat | FIRING   | HighPM25Level            | warning  | sensor-3 | PM2.5 is high in Outdoors (178 µg/m³)                |
| 2026-10-02T16:12:02+00:00 | team-chat | FIRING   | SensorOffline            | warning  | sensor-2 | sensor-2 (Server room) stopped reporting             |
| 2026-10-02T16:11:37+00:00 | team-chat | FIRING   | SensorOverheat           | warning  | sensor-1 | Lab is overheating (43.2 °C)                         |
| 2026-10-02T13:56:45+00:00 | team-chat | RESOLVED | SensorDashboardRestarted | info     | -        | Sensor dashboard restarted (crash or new deployment) |
| 2026-10-02T13:51:45+00:00 | team-chat | FIRING   | SensorDashboardRestarted | info     | -        | Sensor dashboard restarted (crash or new deployment) |

*Figure 11 Notifications delivered to the alert receiver*

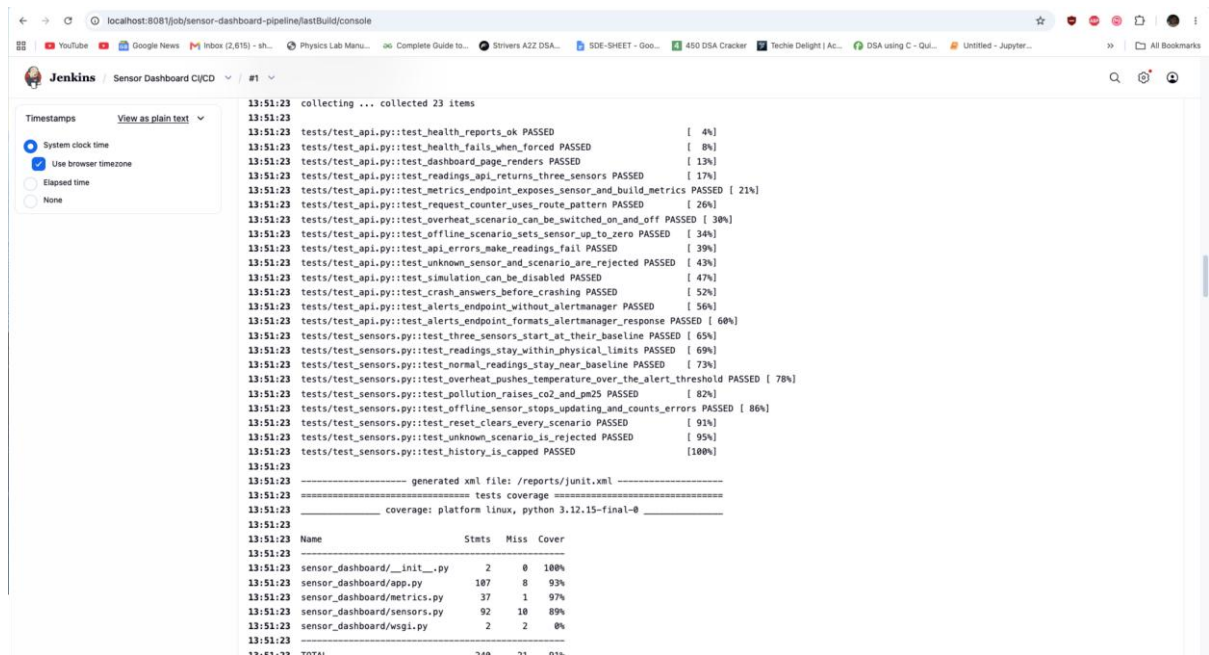
# Testing

## 6.1 Testing Strategy

Testing happens at several levels, and every level runs automatically in both Jenkins and GitHub Actions:

*Table 8 Levels of testing*

| Level                    | What is tested                                                                                                                           | Where it runs                              |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| Lint                     | Code style and common mistakes (flake8)                                                                                                  | Jenkins, GitHub Actions                    |
| Unit tests (23)          | Sensor simulation and failure scenarios, every API endpoint, metrics output, request counting and simulation switches; 91% line coverage | Jenkins (inside Docker), GitHub Actions    |
| Alert rule tests         | Each alert fires at the right time with the right labels and text, and short blips do not fire                                           | Jenkins, GitHub Actions                    |
| Configuration validation | Prometheus config, Alertmanager config, and routing of critical vs warning alerts                                                        | Jenkins, GitHub Actions                    |
| Receiver tests (5)       | Parsing, formatting, HTML escaping and the HTTP endpoints                                                                                | Jenkins, GitHub Actions                    |
| Smoke test               | The deployed container is healthy, is the expected build, serves data and is being scraped                                               | Jenkins after every deploy, GitHub Actions |



*Figure 12 Unit tests passing inside Docker during the Build & Test stage*

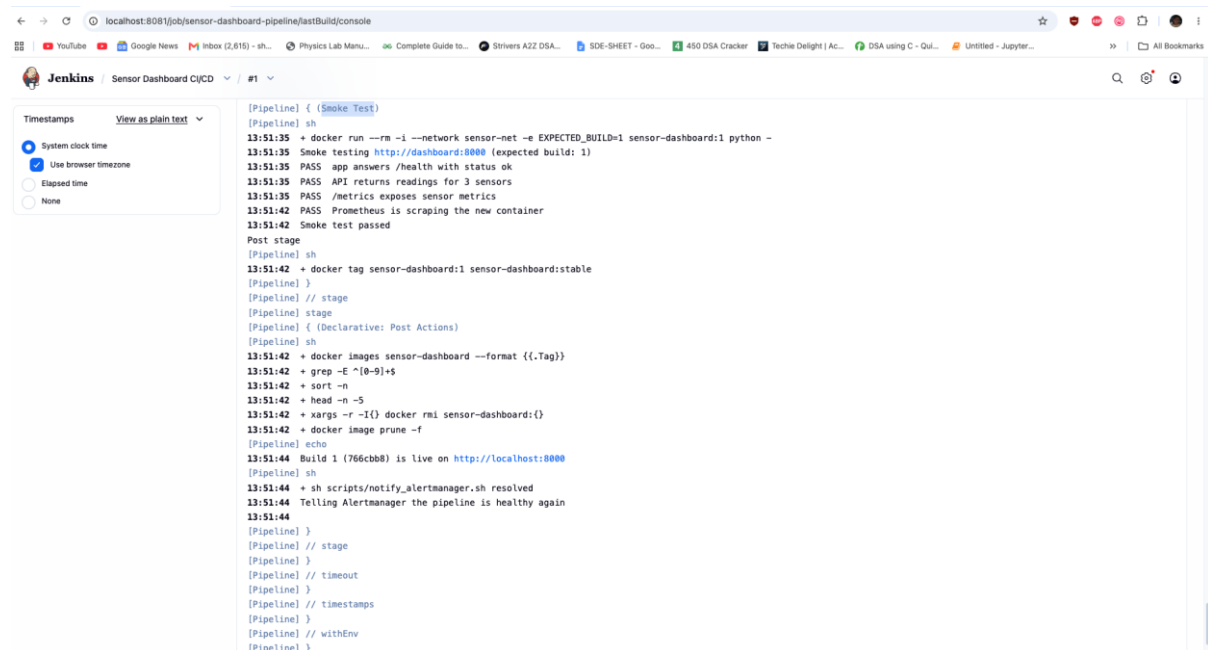
## 6.2 Testing the Alert Rules

Alert rules are code too, and a wrong threshold or a typo means nobody is told when the application fails. The rules therefore have their own unit tests in `monitoring/prometheus/tests/alert_rules_test.yml`, run with `promtool test rules`. Each test feeds Prometheus a synthetic time series and asserts exactly which alerts are firing at a given moment, including their labels and text. One test checks, for example, that a 10-second outage does **not**

page anyone because it is shorter than the `for` duration. The tests were confirmed to be meaningful by changing one expected value and observing the test fail.

## 6.3 Smoke Test

`scripts/smoke_test.py` runs after every deployment from a throwaway container on the `sensor-net` network, the same way Prometheus and users reach the application. It retries each check for up to 45 seconds and verifies that `/health` reports status ok **and the build number that was just deployed** (so an old container still answering cannot pass the test), that the API returns readings for three sensors, that `/metrics` exposes the sensor metrics, and that Prometheus reports the new container as up.



The screenshot shows the Jenkins web interface for a job named 'Sensor Dashboard C/CD'. The console output for the 'Smoke Test' stage is displayed. The output shows the execution of a Docker container and the results of the smoke test.

```
[Pipeline] { (Smoke Test)
[Pipeline] sh
13:51:35 + docker run --rm -i --network sensor-net -e EXPECTED_BUILD=1 sensor-dashboard:1 python -
13:51:35 Smoke testing http://dashboard:8000 (expected build: 1)
13:51:35 PASS app answers /health with status ok
13:51:35 PASS API returns readings for 3 sensors
13:51:35 PASS /metrics exposes sensor metrics
13:51:42 PASS Prometheus is scraping the new container
13:51:42 Smoke test passed
Post stage
[Pipeline] sh
13:51:42 + docker tag sensor-dashboard:1 sensor-dashboard:stable
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
13:51:42 + docker images sensor-dashboard --format {{.Tag}}
13:51:42 + grep -E ^[0-9]+s
13:51:42 + sort -n
13:51:42 + head -n -5
13:51:42 + xargs -r -I{} docker rmi sensor-dashboard:{}
13:51:42 + docker image prune -f
[Pipeline] echo
13:51:44 Build 1 (766cbb8) is live on http://localhost:8000
[Pipeline] sh
13:51:44 + sh scripts/notify_alertmanager.sh resolved
13:51:44 Telling Alertmanager the pipeline is healthy again
13:51:44
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // timeout
[Pipeline] }
[Pipeline] // timestamps
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
```

*Figure 13 Smoke test passing after a deployment*

# Results

## 7.1 Pipeline Runs

The following pipeline runs were performed to demonstrate normal delivery, an automatic rollback and a blocked deployment:

*Table 9 Pipeline runs performed*

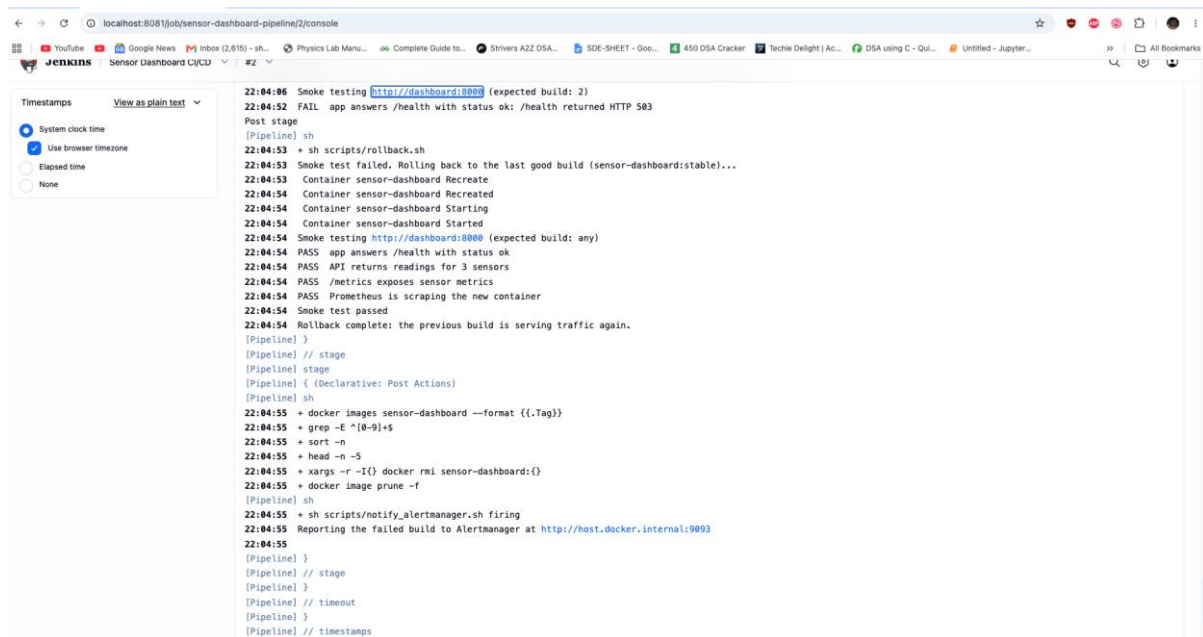
| Build | Trigger                            | What was done                                   | Result                                                                                                                              |
|-------|------------------------------------|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| #1    | Automatic (first poll)             | Normal run                                      | All stages passed in about 34 s; build 1 deployed and tagged stable                                                                 |
| #2    | Manual, SIMULATE_BAD_DEPLOY = true | Deployed a build whose /health returns HTTP 503 | Smoke test failed after retrying for 45 s; build 1 (stable) was restored and verified automatically; PipelineFailed alert sent      |
| #3    | Automatic, after a push            | Commit with a deliberately failing test         | Stopped at Build & Test (1 failed, 22 passed); later stages skipped; the running version was not touched; PipelineFailed alert sent |
| #4    | Automatic, after a push            | Reverted the failing test                       | All stages passed and the new build was deployed; PipelineFailed resolved                                                           |

```

22:14:20 ===== short test summary info =====
22:14:20 FAILED tests/test_sensors.py::test_normal_readings_stay_near_baseline - asser...
22:14:20 ===== 1 failed, 22 passed in 0.48s =====
Post stage
[Pipeline] sh
22:14:20 + mkdir -p reports
22:14:20 + docker cp sensor-tests-3:/reports/junit.xml reports/junit.xml
22:14:20 + docker rm -f sensor-tests-3
22:14:20 + docker rmi sensor-dashboard:test-3
[Pipeline] junit
22:14:20 Recording test results
22:14:20 [Checks API] No suitable checks publisher found.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Validate Monitoring)
Stage "Validate Monitoring" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build Image)
Stage "Build Image" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
Stage "Deploy" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Smoke Test)
Stage "Smoke Test" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // stage

```

*Figure 14 A failing test blocks the deployment*



**Figure 15 Failed smoke test and automatic rollback**

## 7.2 Failure Detection

Each failure was triggered from the dashboard's simulation panel or the `scripts/simulate.sh` script, and the time was measured from triggering the failure to the notification appearing in the alert receiver.

**Table 10 Failure scenarios and detection time**

| Scenario                    | Alert(s) raised                                         | Channel       | Time to notify |
|-----------------------------|---------------------------------------------------------|---------------|----------------|
| Sensor 1 overheats          | SensorOverheat                                          | team-chat     | 30 – 41 s      |
| Sensor 2 goes offline       | SensorOffline                                           | team-chat     | ≈ 30 s         |
| Pollution spike on Sensor 3 | HighCO2Level + HighPM25Level (one grouped notification) | team-chat     | ≈ 45 s         |
| API returns HTTP 500        | HighApiErrorRate                                        | on-call-pager | ≈ 53 s         |
| Application process crashes | Docker restarts it;<br>SensorDashboardRestarted         | team-chat     | ≈ 10 s         |
| Container stopped           | SensorDashboardDown (restart alert inhibited)           | on-call-pager | 44 – 48 s      |
| Container started again     | SensorDashboardDown resolved                            | on-call-pager | 24 – 29 s      |

Each delay is the sum of the scrape interval (5 s), the rule's `for` duration (10 – 30 s), the evaluation interval (5 s) and Alertmanager's `group_wait` (5 s). Shorter durations would notify faster but would also fire on harmless blips; the chosen values suit a demonstration while still filtering out one-off spikes.

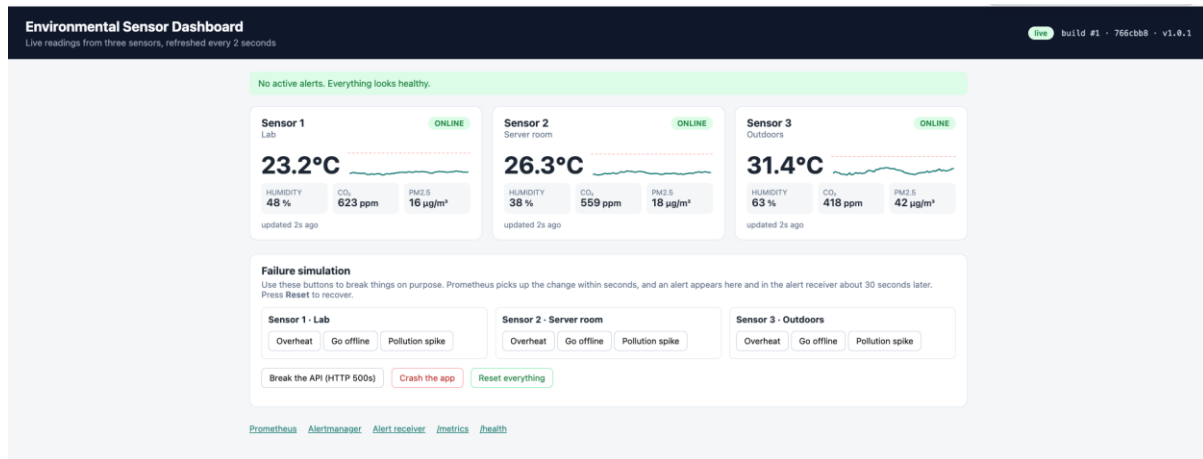


Figure 16 The dashboard in its normal state

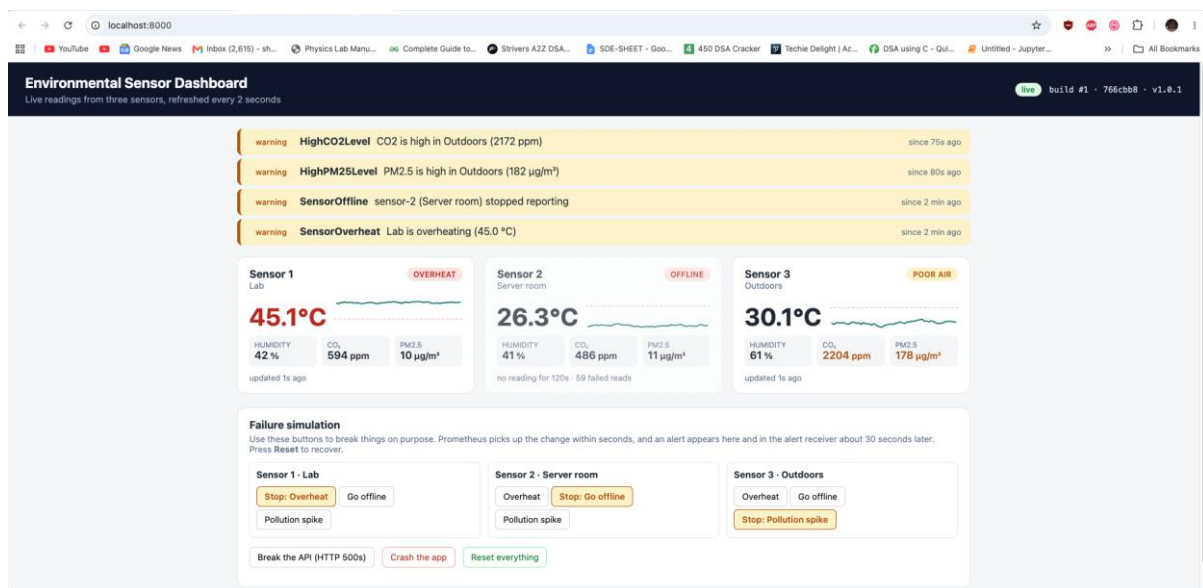


Figure 17 The dashboard during an incident, with alerts shown as a banner

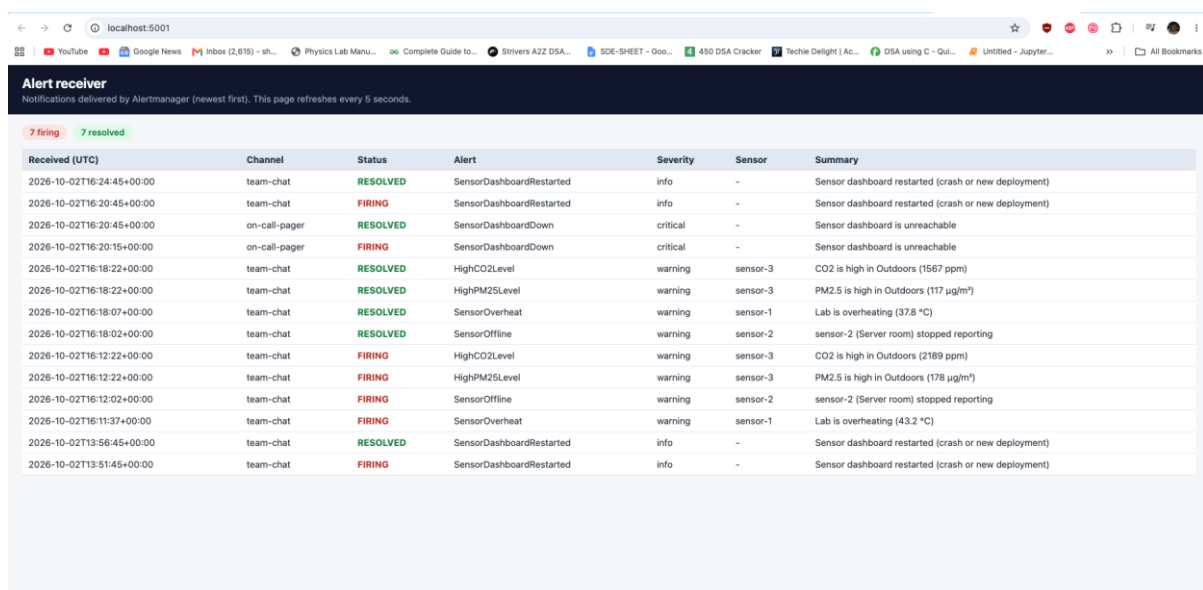


Figure 18 Critical alert routed to the on-call channel

## Critical Analysis

A working demonstration is only part of the story. This chapter covers the problems we ran into, the trade-offs behind the design and the system's limitations.

### 8.1 Challenges Faced and Solutions

*Table 11 Implementation challenges*

| Challenge                                                           | Cause                                                                                                                                   | Solution                                                                                                                            |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Configuration files could not be mounted during Jenkins deployments | Jenkins runs in a container; its workspace path does not exist on the host, so the host's Docker engine cannot bind-mount files from it | Bake prometheus.yml, alert rules and alertmanager.yml into small custom images, which also puts config changes through the pipeline |
| Jenkins could not use the Docker socket                             | The socket belongs to the root group inside the container                                                                               | Add that group with group_add instead of running Jenkins as root; the group id is configurable for Linux hosts                      |
| Alertmanager stayed silent after a restart mid-incident             | It persists which notifications it has already sent and treated the alert as delivered                                                  | Redeploy Alertmanager only when its configuration changes                                                                           |
| Every container restarted on the first Jenkins deploy               | Images built from Jenkins's checkout differed slightly (line endings, timestamps) from those built locally                              | Add .gitattributes to force LF line endings; afterwards only changed services are replaced                                          |
| Port 8080 already in use                                            | Another Jenkins installation on the machine                                                                                             | Make the Jenkins port configurable (JENKINS_PORT) and run on 8081                                                                   |
| Tests could not import the package inside the container             | pytest did not add the working directory to the import path                                                                             | Set pythonpath = . in setup.cfg                                                                                                     |

### 8.2 Design Trade-offs

- **Polling vs webhooks.** Jenkins polls GitHub every minute. A webhook would trigger builds instantly, but GitHub cannot reach a Jenkins server running on a laptop without exposing it to the internet. Polling adds up to a minute of delay in exchange for not opening the machine to inbound traffic.
- **Detection speed vs false alarms.** Short `for` durations (10 – 30 s) give fast demonstrations. In production 2 – 5 minutes is more common, because a short spike in CPU, network or sensor noise should not page someone at night.
- **Docker socket access.** Mounting the Docker socket gives Jenkins full control of the host's Docker engine, which is effectively root access. It is acceptable on a lab machine but would be replaced by dedicated build agents in production.
- **In-memory state.** Keeping sensor state in memory keeps the application simple, but it forces a single worker process and loses state on restart.

### 8.3 Limitations

- The sensor data is simulated; there is no real device communication, network unreliability or data persistence.

- Everything runs on one host, so the host itself is a single point of failure and the monitoring cannot report a failure of the machine it runs on.
- Deployment replaces the container in place, so there is a few seconds of downtime on every release; there is no rolling or blue-green deployment.
- Images are built and stored locally rather than pushed to a registry, so the deployment cannot move to another machine without rebuilding.

## **8.4 Future Improvements**

- Replace the simulator with real sensors, for example an ESP32 publishing readings over MQTT, without changing the pipeline or the monitoring.
- Add Grafana dashboards on top of Prometheus for long-term trends and capacity planning.
- Deliver notifications to Slack, e-mail or Telegram by changing only the Alertmanager receivers.
- Push images to a registry (for example Docker Hub or Nexus) and add an image vulnerability scan such as Trivy as a pipeline stage.
- Use a GitHub webhook through a public Jenkins URL instead of polling, and run builds on separate Jenkins agents.
- Deploy to Kubernetes for rolling updates with zero downtime, self-healing and horizontal scaling.

## Conclusion

This project covers the full DevOps loop for a containerized application: code, build, test, deploy, monitor and alert. Every change goes through version control and an automated Jenkins pipeline that refuses to deploy untested code, verifies each deployment with a smoke test and undoes a bad release on its own. Docker makes the application behave the same way on a developer's laptop, on GitHub's build machines and in deployment, so the image that was tested is the one that runs.

Prometheus and Alertmanager turned every failure into a grouped, correctly routed notification within roughly half a minute, held back the side-effect alerts during a bigger outage, and confirmed when each problem was fixed. Because the monitoring configuration is also code, with its own tests and validation in the pipeline, the alerting is checked as carefully as the application it watches.

Every behaviour described in this report was shown with a real, deliberately injected failure. The project shows how a small set of widely used open-source tools, combined with practices such as pipeline as code, configuration as code, immutable versioned images and automated rollback, can make software delivery both faster and safer.

## References

1. Jenkins Project. *Pipeline Syntax*. Jenkins User Documentation. <https://www.jenkins.io/doc/book/pipeline/syntax/>
2. Jenkins Project. *Configuration as Code Plugin*. <https://www.jenkins.io/projects/jcasc/>
3. Docker Inc. *Multi-stage builds* and *Docker Compose* documentation. <https://docs.docker.com/>
4. Prometheus Authors. *Configuration, Alerting rules and Unit testing for rules*. <https://prometheus.io/docs/>
5. Prometheus Authors. *Alertmanager: routing, grouping and inhibition*. <https://prometheus.io/docs/alerting/latest/alertmanager/>
6. Prometheus Authors. *prometheus\_client: Python client library*. [https://github.com/prometheus/client\\_python](https://github.com/prometheus/client_python)
7. Pallets Projects. *Flask Documentation*. <https://flask.palletsprojects.com/>
8. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media (Chapter 6: Monitoring Distributed Systems).
9. Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps Handbook*. IT Revolution Press.
10. Project source code: [https://github.com/Yashvi2874/DevOps\\_failure\\_monitoring](https://github.com/Yashvi2874/DevOps_failure_monitoring)